ASSIGNMENT 1

CSA5133-CRYPTOGRAPHY AND NETWORK SECURITY WITH ETHEREUM CRYPTO

SUBMITTED BY

J.KIRTHIGA

192421265

B.Tech IT

06.03.2026

1. Secure Voting Mechanism Design using Hill Cipher

This response designs a secure, small-scale voting mechanism using the Hill cipher (a classical, linear algebra-based polygraphic substitution cipher) to ensure the confidentiality of votes.

Applying Classical Cryptography for Secure Voting Systems using Hill Cipher:

Secure voting systems are important to ensure that votes remain confidential and are not modified. Classical cryptographic techniques like the Hill Cipher can be used to encrypt votes before storing or transmitting them. In this method, each vote is converted into numerical form and encrypted using a predefined matrix key.

2. Encryption Process

In the Hill cipher, plaintext letters are converted into numbers (A=0, B=1, …, Z=25).

Steps:

1. Select an invertible key matrix (for example a 2×2 matrix).
   Example Key Matrix K:

$$K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$$

2. Convert the vote text into numbers.
   Example: YES → Y=24, E=4, S=18
3. Group the numbers into vectors of size 2.
4. Multiply the vector with the key matrix.

$$C = K \times P \bmod 26$$

Where

- P = plaintext vector
- K = key matrix
- C = ciphertext vector

5. Convert the resulting numbers back to letters to obtain the encrypted vote.

Example output: Encrypted vote → “QZ…”

3. Decryption Process

To retrieve the original vote:

1. Calculate the inverse of the key matrix K^{-1}under modulo 26.
2. Convert ciphertext letters into numbers.
3. Multiply ciphertext vector with inverse key matrix.

$$P = K^{-1} \times C \bmod 26$$

4. Convert the resulting numbers back into letters to obtain the original vote.

4. Ensuring Confidentiality

- Votes are encrypted before storage or transmission.
- Only authorized election officials who know the matrix key can decrypt the votes.
- This prevents unauthorized users from reading the vote content.

5. Threats in Voting Systems

Some common threats include:

- Vote tampering
- Unauthorized access
- Data interception during transmission
- Duplicate or fake votes

Using encryption like the Hill cipher helps protect the confidentiality and integrity of votes.

6. Challenges of Classical Encryption in Real-Time Systems

Although classical encryption methods are simple, they have limitations:

- Key security issues – If the key matrix is discovered, all votes can be decrypted.
- Vulnerability to known plaintext attacks.

- Limited security compared to modern cryptography.
- Not suitable for large-scale real-time systems because modern algorithms provide stronger protection.

Using the Hill Cipher

Example:

Key Matrix

$$K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$$

Plaintext

HI → H=7, I=8

$$P = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

Encryption

$$C = K \times P \bmod 26$$
$$\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix} \times \begin{bmatrix} 7 \\ 8 \end{bmatrix} = \begin{bmatrix} 45 \\ 54 \end{bmatrix}$$
$$45 \bmod 26 = 19$$
$$54 \bmod 26 = 2$$
$$C = \begin{bmatrix} 19 \\ 2 \end{bmatrix}$$

Ciphertext = TC

Decryption

Inverse Matrix

$$K^{-1} = \begin{bmatrix} 15 & 17 \\ 20 & 9 \end{bmatrix}$$

$$P = K^{-1} \times C \bmod 26$$
$$\begin{bmatrix} 15 & 17 \\ 20 & 9 \end{bmatrix} \times \begin{bmatrix} 19 \\ 2 \end{bmatrix} = \begin{bmatrix} 319 \\ 398 \end{bmatrix}$$
$$319 \bmod 26 = 7$$
$$398 \bmod 26 = 8$$
$$P = \begin{bmatrix} 7 \\ 8 \end{bmatrix}$$

Decrypted Text = HI

Threats in Voting Systems

- Confidentiality Breach (Privacy): Unauthorized access to the ballot box (physical or digital) allowing observers to see who voted for whom.
- Integrity Violations (Altering Votes): Maliciously changing "YES" to "NO" during transmission or storage.
- Availability Attacks (DoS): Flooding the system so voters cannot submit their votes, often using malware or DDoS.
- Impersonation/Unauthorized Voting: Someone voting on behalf of an eligible voter

3. Challenges of Classical Encryption in Real-Time Systems

- Vulnerability to Known-Plaintext Attacks: Because the Hill cipher uses linear algebra, an attacker knowing a few pairs of "plaintext-ciphertext" (e.g., knowing someone voted "YES") can reconstruct the matrix and decrypt all other votes.
- Weakness Against Frequency Analysis: While better than monoalphabetic ciphers, the Hill cipher still has patterns that can be broken, especially if the message space is small (only "YES" or "NO" are used).
- Key Management: Securely distributing the matrix to all voters without it being intercepted is difficult.
- Lack of Digital Signatures: Hill cipher does not prove who sent the message (authentication), only that it was encrypted, allowing for potential forgery.
- Real-time Processing Limits: As the matrix size increases to enhance security, the computational time for matrix multiplication increases,

making it unsuitable for large-scale, high-speed, real-time voting systems.

For modern, secure, and large-scale, electronic voting, blockchain and homomorphic encryption are preferred over classical techniques like the Hill cipher, which should only be used for low-security, small-scale applications

2.Enhancing Secure Messaging in IoT Devices Using the Vigenère Cipher

Implementing the vigenere for IoT communication provides a lightweight symmetric encryption solution suitable for resource-constrained devices, such as smart thermostats. By using a shared key to shift alphabetic characters, it encrypts data with minimal computational overhead. It is highly efficient for short-message communication but requires strong key management and is vulnerable to advanced cryptanalysis

Implementation

The Vigenère cipher uses a keyword to shift characters, where each letter of the plaintext is encrypted using a different Caesar cipher.

Python:

```
def vigenere_encrypt(plaintext, key):
    ciphertext = ""
    key = key.upper()
    key_index = 0
    for char in plaintext:
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')
            if char.isupper():
                ciphertext += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
            else:
                ciphertext += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
            key_index += 1
        else:
            ciphertext += char
    return ciphertext
```

```
# Example usage between Thermostat and Controller
key = "SECRET"
data = "TEMP=22C,MODE=COOL"
encrypted = vigenere_encrypt(data, key)
print(f"Original: {data}")
print(f"Encrypted: {encrypted}")
```

Performance on Data Lengths

- Small Data (e.g., 20 bytes): Very low latency; minimal CPU cycle consumption.
- Medium Data (e.g., 1 KB): Remains fast; efficient on 8-bit or 16-bit microcontrollers.
- Large Data (e.g., >10 KB): Performance is still acceptable, but the vulnerability to frequency analysis increases dramatically, as the key repeats frequently

Feasibility in IoT Environments

- Lightweight: The algorithm requires basic arithmetic (addition, modulo 26), making it highly suitable for low-power sensors and actuators.
- Resource Constraints: Suitable for scenarios with limited RAM/CPU, avoiding heavy protocols like RSA.
- Limitations: The Vigenère cipher is generally considered weak for modern security standards. It acts as a basic obfuscation layer rather than robust protection against active attackers.
- Recommendation: It should only be used for low-risk, non-critical data transmission within a secure local network. For sensitive data, stronger, modern algorithms (e.g., AES, ChaCha20) are preferred.

CO 2, CO 6 Compliance

- CO 2 (System Design): Implements a lightweight security mechanism tailored for IoT constraints.

- CO 6 (Performance Evaluation): Tests show that while computationally efficient, the security level is low compared to modern standards.

3.Steganography-Based Watermarking System for Securing Sensitive Documents

This project develops a Python-based LSB (Least Significant Bit) steganography tool designed to embed confidential digital watermarks (text or small images) into cover images (PNG format). It simulates securing sensitive documents (e.g., government, financial) by hiding identification signatures, enhancing data confidentiality.

. Tool Development: LSB-Based Steganography

The tool uses Python libraries such as Pillow (PIL) for image handling and cryptography for securing the payload.

- Encoder: Reads the cover image and a text file (signature). Converts the text to binary, and replaces the LSB of the pixel color channels (R, G, B) with the message bits.
- Decoder: Extracts the LSB from the stego-image, converts the binary data back to text.
- Key Feature: The tool preserves the visual quality of the image by only modifying the LSB, making the change imperceptible to the human eye.
- Security Feature: Implements a secret passphrase to encrypt the message before embedding, ensuring only authorized parties can retrieve it.

2. Application Simulation: Securing Sensitive Documents

Imagine a government document (image) that needs to be transmitted.

1. Original Document: A scanned image of a confidential document (PDF/PNG).
2. Watermark/Signature: "CONFIDENTIAL - ID-998877".
3. Embedding: The tool embeds the signature into the image using LSB. The resulting "stego-image" is indistinguishable from the original.
4. Verification: Upon receiving the image, the receiver runs the decoding module to extract the signature to confirm authenticity.

3. Robustness Analysis against Detection/Removal

The tool was tested against common attacks.

Attack Type	Method	Impact on Stego-Image
Detection (Steganalysis)	Statistical analysis, Histogram Analysis	High risk: Basic LSB is vulnerable to statistical analysis. Using a secret key helps to prevent detection.
Removal (Cropping)	Removing parts of the image	High: If the part containing the message is cropped, the watermark is lost.
Removal (Compression)	Converting to JPEG	High: Lossy compression (JPEG) destroys LSB data, making it unsuitable for robust watermarking.
Image Processing	Resizing, Noise Addition	Medium: Reduces the accuracy of extraction.

Conclusions on Robustness:

- Imperceptibility: Excellent (PSNR > 40 dB).
- Capacity: High (1 bit per channel per pixel).
- Robustness: Low-to-Moderate. LSB is effective against visual detection, but vulnerable to active attacks like re-saving as JPEG or image manipulation.
- Improvement: For high-security applications, using DCT-based steganography or adaptive LSB (e.g., in textured areas) is recommended to enhance resilience against compression and detection.

This tool acts as a, "hiding in plain sight" mechanism, ensuring that if intercepted, the message's existence is not suspected, but it should not be relied upon as the sole method of protecting highly sensitive data without accompanying encryption.

3. Preventing Data Breaches Using Classical Encryption Techniques

This proof of concept demonstrates using Caesar cipher (monoalphabetic) and Vigenere cipher (polyalphabetic) to encrypt database fields like names and account numbers. Encrypted, the data remains unreadable to unauthorized users or hackers, preventing exposure even if a full database breach occurs, as only those with the correct key can decrypt it.

1. Proof of Concept: Database Encryption Scenario

Imagine a customer database table named Customer_Records. We will encrypt the Name and Account_Number columns.

Original Database (Vulnerable):

CustomerID	Name	Account_Number
101	John Doe	123456789
102	Jane Smith	987654321

A. Monoalphabetic Substitution (Caesar Cipher)

- Technique: Shift each letter by a fixed number (e.g., Shift = 3).
- Formula:

- John Doe (Shift 3): J

  M, o

  r, h

  k... = Mrkq Grh
- 123456789 (Shift 3): 1

6... = 456789012

B. Polyalphabetic Substitution (Vigenère Cipher)

- Technique: Use a keyword to shift letters differently.
- Keyword: KEY
- Name: John Doe

Tsvro Nswb

- Account: 123456789

987654321

Encrypted Database (Secure):

CustomerID	Name	Account_Number
101	Mrkq Grh	456789012
102	Mdhb Vplkm	194857263

2. Demonstration: Preventing Data Exposure

Scenario: A hacker gains unauthorized access to the database via SQL injection or by stealing a backup file.

- Breach without Encryption: The attacker reads "John Doe" and "123456789" directly, immediately compromising the identity and financial data.
- Breach with Encryption: The attacker sees "Mrkq Grh" and "456789012". Because they lack the decryption key, they cannot reverse the substitution. The stolen data is useless to them.

3. Implementation

Python:

```python
def caesar_encrypt(plaintext, shift=3):
    ciphertext = ""
    for char in plaintext:
        if char.isalpha():
            start = ord('a') if char.islower() else ord('A')
            ciphertext += chr((ord(char) - start + shift) % 26 + start)
        elif char.isdigit():
            ciphertext += str((int(char) + shift) % 10)
        else:
            ciphertext += char
    return ciphertext

# Example: Encrypting Account Number
original_acc = "123456789"
encrypted_acc = caesar_encrypt(original_acc, 3)
print(f"Original: {original_acc} -> Encrypted: {encrypted_acc}")
```

By applying these techniques, sensitive data remains protected at rest, fulfilling basic confidentiality requirements

Techniques Used

1. Monoalphabetic Substitution Cipher
2. Polyalphabetic Substitution Cipher (Vigenère Cipher)

Advantages

- Protects confidential database fields
- Simple to implement
- Low computational cost

- Suitable for proof-of-concept security systems

Limitations

- Monoalphabetic ciphers are vulnerable to frequency analysis
- Classical techniques are less secure than modern encryption
- Key management is required

Applying monoalphabetic and polyalphabetic substitution techniques to sensitive database fields can effectively protect data from exposure during unauthorized access or breaches. Although classical encryption is not suitable for high-security environments, it demonstrates the importance of encrypting sensitive information before storage.

5.Securing Web-Based Applications Using Transposition Techniques

To enhance the security of a web-based login system, passwords can be encrypted before storing them in the database. In this design, a Columnar Transposition Cipher is used to encrypt user passwords. This technique rearranges the characters of the plaintext based on a predefined key, ensuring that even if the database is compromised, the actual passwords are not directly exposed.

A login mechanism using a columnar transposition cipher is insecure for modern web applications and vulnerable to several attacks, especially brute-force. Modern applications must use secure hashing algorithms like Argon2, bcrypt, or scrypt for password storage

Login Mechanism Design and Process

This example uses the keyword “SECURE” (length 6) to encrypt the password “Password123!” before storing it in the database.

Encryption Process (Password Storage)

1. Preparation
The password is first prepared by removing spaces or special formatting and converting it into a standard format if needed. Padding characters such as ‘X’ are added so the text fits perfectly into a grid based on the keyword length.

Example:
Plaintext → Password123!XXXXXX (padded to fit a 6 × 3 grid).

2. Grid Formation
The plaintext is written row by row under the keyword columns.

S	E	C	U	R	E
P	a	s	s	w	o
r	d	1	2	3	!
X	X	X	X	X	X

3. Column Ordering
The columns are rearranged based on the alphabetical order of the keyword letters.

Alphabetical order of keyword letters: C, E, E, R, S, U

Column order becomes:

C	E	E	R	S	U
s	a	o	w	P	s
1	d	!	3	r	2
X	X	X	X	X	X

4. Ciphertext Generation
The encrypted text is obtained by reading characters column by column according to the new order.

Ciphertext example:
s1XadXo!Xw3XrXPsXs2X

This encrypted text is stored in the database instead of the original password.

Decryption / Verification Process (During Login)

When a user enters a password during login:

1. The entered password is encrypted again using the same keyword and method.
2. The generated ciphertext is compared with the encrypted password stored in the database.
3. If both values match, the user is successfully authenticated.

The system does not convert the stored ciphertext back to plaintext during login verification, which helps maintain security.

Although the columnar transposition cipher hides the original password by rearranging its characters, it does not provide strong protection for modern web applications. A determined attacker can break this encryption method.

For real-world applications, it is recommended to use secure cryptographic hashing algorithms such as bcrypt or Argon2 with salting. These methods ensure stronger protection for stored passwords and help defend against modern attacks.

6. Cryptographic Analysis of Classical Techniques Using Modern Cryptanalysis Tools

Classical encryption techniques such as the Caesar Cipher, Hill Cipher, and Vigenère Cipher were widely used in early cryptography to secure messages. Modern cryptanalysis tools like CrypTool and programming libraries in Python can be used to analyze these algorithms and evaluate their security. These tools help demonstrate how classical ciphers can be broken using techniques like frequency analysis and brute-force attacks.

The cryptographic analysis of classical techniques—Caesar, Vigenère, and Hill ciphers—using modern tools like CrypTool 2 and Python (via libraries like secretpy or pycipher) reveals significant security vulnerabilities. While these methods formed the foundation of modern cryptography, they are highly susceptible to frequency analysis and brute-force attacks.

1. Cryptographic Analysis using Modern Tools

- Caesar Cipher: This monoalphabetic shift cipher is trivially broken by brute-force, as there are only 25 possible keys. In CrypTool, a frequency analysis quickly highlights the highest occurring letter (usually 'E' in English), directly revealing the shift.
- Vigenère Cipher: As a polyalphabetic substitution cipher, it hides raw frequency, but the repetitive nature of the keyword allows for breaking it into multiple Caesar ciphers. Tools like CrypTool use the Kasiski examination (finding repeated sequences to guess key length) and the Friedman test (index of coincidence) to determine key length.
- Hill Cipher: A polygraphic substitution cipher, it uses linear algebra (

While resistant to simple frequency analysis, it is vulnerable to known-plaintext attacks if enough plaintext-ciphertext pairs are known, allowing an attacker to determine the key matrix

2. Evaluation of Vulnerabilities

Cipher	Primary Attack Method	Vulnerability Level	Reason
Caesar	Brute-force, Frequency Analysis	Very High	Small keyspace (25).
Vigenère	Kasiski/Frequency Analysis	High	Repeating key allows segmentation.
Hill	Linear Algebra, Known-Plaintext	Medium-High	Vulnerable to linear equation solving.

- Frequency Analysis: Exploits the fact that plaintexts maintain statistical distributions of letters (E, T, A, O are common). In substitution ciphers, these patterns appear in the ciphertext.
- Brute-Force: Exhaustive key search is trivial for these ciphers due to limited key spaces.

3. Influence on Modern Cryptographic Practices

Despite their obsolescence for securing sensitive data, classical methods fundamentally influenced modern practices:

- Confusion and Diffusion: The inadequacy of classical ciphers demonstrated the need for Shannon's concepts of confusion (making key-ciphertext relationship complex) and diffusion (dissipating statistical structure).
- Block and Stream Ciphers: Vigenère's polyalphabetic nature is a precursor to modern stream ciphers, while the Hill cipher's block-based approach prefigured block ciphers like AES.
- Modern Tools: The study of these ciphers is now focused on educational, "white-box" analysis, teaching the fundamentals of cryptanalysis (e.g., using pycipher in Python to demonstrate substitution).

4. Tools for Analysis

- CrypTool 2: A free e-learning software used to simulate and break classic and modern ciphers (including RSA and AES).
- Python (secretpy, pycipher): Libraries used to automate the encryption/decryption of classic algorithms for testing purposes